

Blocco 13 - Fondamenti NoSQL

Perché nascono i database non relazionali e quali famiglie usare

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Perché NoSQL

NoSQL non sostituisce il modello relazionale: risponde a problemi in cui forma, volume o velocità del dato cambiano le priorità.

Obiettivo del blocco

Problema

Dopo aver visto DBMS relazionali, SQL e dashboard, serve capire quando un sistema dati non parte da tabelle, join e schema rigido.

Idea guida

La scelta del database dipende da access pattern, forma del dato, frequenza di scrittura, coerenza richiesta e costo operativo.

Esempio motivante: telemetria

Un sensore può inviare messaggi come questi:

- ▶ un trasformatore invia temperatura, carico, tensione;
- ▶ una centralina ambientale invia PM2.5, umidità, vento;
- ▶ uno smart meter invia consumo, tensione e corrente;
- ▶ una nuova versione firmware aggiunge campi non previsti.

In un modello relazionale possiamo gestire tutto, ma dobbiamo decidere subito molte tabelle, colonne e relazioni. In NoSQL spesso partiamo dal documento ricevuto.

La tabella non è sempre il punto di partenza

Domanda	Effetto sul modello
Il dato ha campi variabili?	documento JSON o colonne flessibili
Serve latenza molto bassa per chiave?	key-value store
Serve scrivere serie temporali continue?	time-series o wide-column
Serve attraversare relazioni profonde?	graph database
Serve ricerca testuale o log analytics?	motore search/documentale

Non è una gara SQL contro NoSQL

- ▶ SQL è forte quando relazioni, vincoli, transazioni e interrogazioni ad hoc sono centrali.
- ▶ NoSQL è utile quando la forma del dato è variabile, il carico è distribuito o la query principale è già nota.
- ▶ Molti sistemi reali usano entrambi: PostgreSQL per dati core, MongoDB per eventi/documenti, Redis per cache, Elasticsearch per ricerca.

Analogia

Un foglio contabile, un diario JSON, una rubrica indicizzata e una mappa stradale possono descrivere la stessa realtà. Cambia il tipo di domanda che vogliamo fare velocemente.

Dalla normalizzazione all'aggregate

Nel modello relazionale scomponiamo per ridurre ridondanza e anomalie.

Nei database documentali spesso ragioniamo per **aggregate**: un oggetto letto e scritto insieme.

- ▶ ordine con righe ordine;
- ▶ profilo cliente con preferenze;
- ▶ evento telemetrico con metriche e metadati;
- ▶ ticket con snapshot del canale di provenienza.

La domanda diventa: “quale informazione deve arrivare insieme, nello stesso documento, per servire la query più frequente?”

Documento JSON: esempio

```
{
  "device_id": "TR-roma-001",
  "ts": "2026-05-13T08:10:00Z",
  "device_type": "transformer",
  "site": "Roma Nord",
  "metrics": {
    "temperature_c": 71.4,
    "load_kw": 910,
    "voltage_v": 19950
  },
  "status": "warning"
}
```


Schema-on-write e schema-on-read

Schema-on-write

La struttura viene controllata prima della scrittura. È l'approccio tipico dei database relazionali.

Schema-on-read

La struttura viene interpretata quando leggiamo il dato. È frequente in sistemi documentali, log, data lake e pipeline analitiche.

La libertà non elimina la disciplina: sposta parte della responsabilità su validazione applicativa, convenzioni, indici e pipeline di pulizia.

2. Famiglie principali

NoSQL è un'etichetta ampia: contiene tecnologie con modelli dati e compromessi molto diversi.

Famiglia 1: document database

- ▶ modello dati: documenti JSON/BSON;
- ▶ esempi: MongoDB, Couchbase;
- ▶ punto forte: oggetti complessi, schema flessibile, sviluppo rapido;
- ▶ attenzione: duplicazione, aggiornamenti multi-documento, query aggregate da progettare;
- ▶ uso tipico: profili, cataloghi, eventi, telemetria, configurazioni.

Famiglia 2: key-value store

- ▶ modello dati: chiave e valore opaco;
- ▶ esempi: Redis, DynamoDB in uso key-value, Memcached;
- ▶ punto forte: latenza bassa e accesso diretto;
- ▶ attenzione: query limitate, modello molto guidato dall'applicazione;
- ▶ uso tipico: cache, sessioni, rate limiting, feature flags, code leggere.

Famiglia 3: wide-column

- ▶ modello dati: righe distribuite per chiave, colonne flessibili;
- ▶ esempi: Cassandra, HBase, Bigtable;
- ▶ punto forte: grandi volumi, scritture distribuite, alta disponibilità;
- ▶ attenzione: query da progettare prima, join assenti o scoraggiati;
- ▶ uso tipico: IoT, clickstream, log, metriche operative su scala elevata.

Famiglia 4: graph database

- ▶ modello dati: nodi, archi, proprietà;
- ▶ esempi: Neo4j, JanusGraph;
- ▶ punto forte: attraversamenti e relazioni profonde;
- ▶ attenzione: non è la scelta naturale per semplici aggregazioni tabellari;
- ▶ uso tipico: frodi, reti, supply chain, raccomandazioni, dipendenze infrastrutturali.

Famiglia 5: search e time-series

- ▶ search: Elasticsearch, OpenSearch, Solr;
- ▶ time-series: InfluxDB, TimescaleDB, Prometheus, MongoDB time-series collections;
- ▶ punto forte: ricerca, log analytics, metriche temporali;
- ▶ attenzione: retention, cardinalità, downsampling e costo storage;
- ▶ uso tipico: osservabilità, monitoraggio, telemetria, audit log.

Confronto sintetico

Famiglia	Quando aiuta	Rischio principale
Document	dati gerarchici e variabili	documenti troppo grandi o duplicati
Key-value	lookup per chiave	query quasi inesistenti
Wide-column	scritture massive distribuite	modello dipendente dalle query note
Graph	relazioni profonde	costo se servono solo aggregazioni
Search/time-series	log, testo, metriche	retention e cardinalità

3. Coerenza e distribuzione

La scalabilità non è gratis: si paga scegliendo quale garanzia rilassare e dove mettere complessità.

Replica e partizionamento

- ▶ replica: più copie dello stesso dato per disponibilità e letture;
- ▶ partizionamento: dati divisi su nodi diversi;
- ▶ sharding key: criterio con cui una riga/documento finisce su una partizione;
- ▶ hot partition: troppe scritture finiscono sulla stessa chiave;
- ▶ rebalancing: spostamento dati quando il cluster cambia.

Un database distribuito non è solo un database più grande: è un sistema in cui le scelte di chiave diventano scelte architetturali.

CAP, in modo operativo

Il teorema CAP è spesso raccontato male. In pratica, quando la rete si partiziona, dobbiamo scegliere cosa privilegiare:

- ▶ **consistency**: tutti leggono subito lo stesso valore;
- ▶ **availability**: il sistema continua a rispondere;
- ▶ **partition tolerance**: il sistema sopporta problemi di rete.

Non significa scegliere una lettera per sempre. Significa capire quali operazioni tollerano ritardi, quali no, e quale dato è critico.

Eventual consistency

Definizione pratica

Dopo una scrittura, copie o viste diverse possono non essere immediatamente allineate, ma convergono se non arrivano altri aggiornamenti.

Esempi accettabili:

- ▶ contatore visualizzazioni;
- ▶ feed eventi;
- ▶ metriche telemetriche aggregate;
- ▶ cache di una dashboard.

Esempi meno accettabili: saldo contabile, inventario critico, autorizzazioni di sicurezza.

Molti database NoSQL moderni offrono forme di transazione, ma il punto non è solo la presenza della funzione.

- ▶ Una transazione multi-documento può costare più di un documento ben modellato.
- ▶ Una lettura veloce può richiedere duplicazione controllata.
- ▶ Un aggiornamento asincrono può essere corretto se il dominio lo permette.

Nel dubbio, prima si chiarisce la garanzia richiesta dal processo, poi si sceglie la tecnologia.

4. Esempi motivati

La scelta NoSQL si capisce meglio partendo da scenari concreti, non da definizioni astratte.

Scenario A: catalogo prodotti

- ▶ ogni categoria ha attributi diversi;
- ▶ le schede prodotto vengono lette spesso come oggetto completo;
- ▶ le variazioni di schema sono frequenti;
- ▶ una soluzione documentale evita molte tabelle attributo-valore.

Attenzione: se il problema principale è contabilità, fatturazione o stock transazionale, il database relazionale resta centrale.

Scenario B: sessioni utente

- ▶ accesso quasi sempre per session id;
- ▶ scadenza automatica;
- ▶ latenza molto bassa;
- ▶ valore letto e scritto come blocco unico.

Un key-value store è naturale. Non serve costruire un modello ricco se la domanda è sempre: “dammi il valore associato a questa chiave”.

Scenario C: manutenzione infrastrutturale

- ▶ dispositivi, sottostazioni, linee, dipendenze e tratte;
- ▶ domanda frequente: “se questo nodo fallisce, quali elementi dipendono da lui?”;
- ▶ attraversamenti multi-hop;
- ▶ un graph database può rendere la domanda più diretta.

Scenario D: telemetria

- ▶ molte scritture append-only;
- ▶ dati con metriche diverse per tipo dispositivo;
- ▶ analisi per finestra temporale;
- ▶ retention e aggregazioni periodiche;
- ▶ possibile uso di document/time-series store.

Questo scenario diventa il laboratorio dei blocchi 15-16.

5. Metodo di scelta

Prima della tecnologia, si scrive la scheda decisionale del caso d'uso.

- ➊ Qual è l'unità letta più spesso?
- ➋ Quali query devono essere veloci?
- ➌ Il dato cambia struttura spesso?
- ➍ Serve transazione forte tra oggetti diversi?
- ➎ Qual è il volume di scrittura?
- ➏ Quale ritardo di consistenza è accettabile?
- ➐ Come si cancellano o archiviano i dati vecchi?

Errore frequente: scegliere per moda

Segnale di rischio

“Usiamo NoSQL perché scala” è una frase incompleta.

Bisogna dire:

- ▶ quale carico deve scalare;
- ▶ quale query deve migliorare;
- ▶ quale garanzia si può rilassare;
- ▶ quale complessità applicativa siamo disposti a gestire.

- ▶ NoSQL indica famiglie diverse, non una singola tecnologia.
- ▶ La scelta parte da access pattern e garanzie richieste.
- ▶ Document, key-value, wide-column, graph, search e time-series risolvono problemi diversi.
- ▶ La telemetria è un caso adatto per discutere documenti, metriche variabili e aggregazioni temporali.